Uninformed search



Zola Mahlaza

Department of Computer Science

Feb 2026

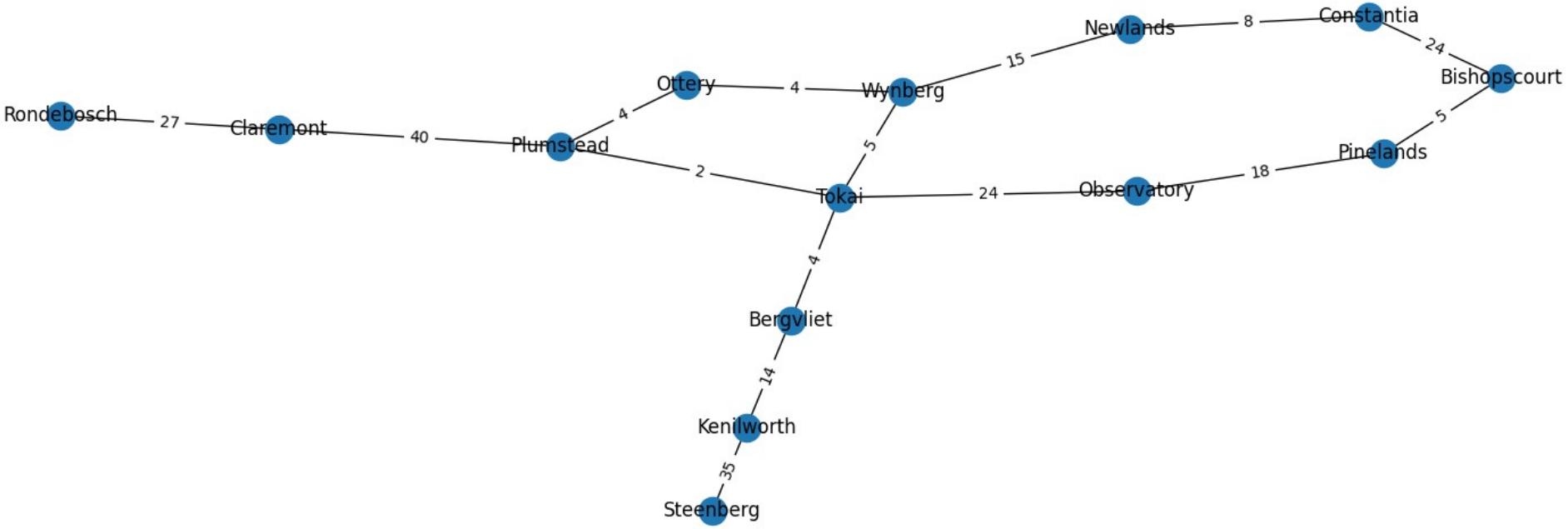
Topics

- Overview of uninformed search strategies
  - To explore search algorithms without heuristic information
- Search task, analyze redundant paths, compare performance metrics, and discuss algorithms

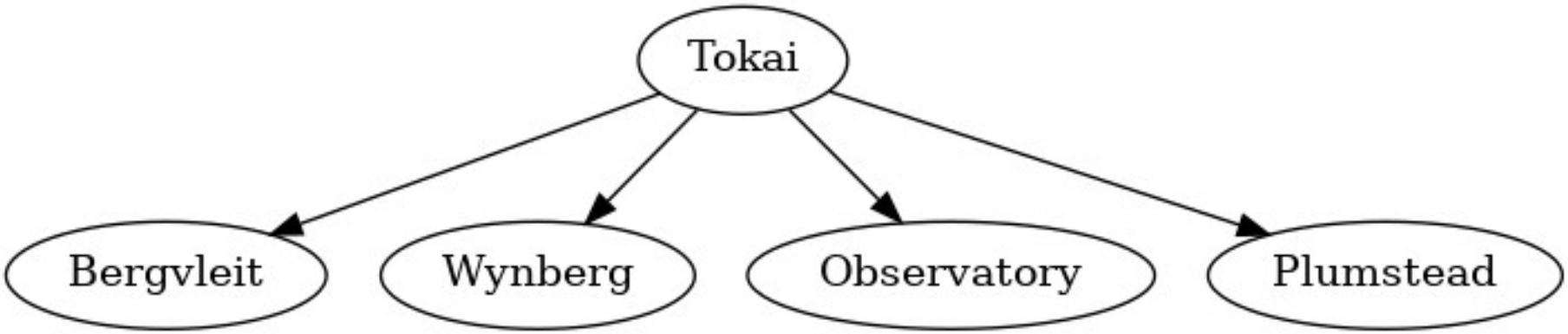
Search task

- A search task involves finding a solution from a starting state to a goal state
- Uninformed search avoids heuristic information
- Capture state-space as a search tree
  - Space contains all possible states and the actions
  - Tree captures actual paths explored to reach a goal
- Some states are duplicated in tree
- Path to root is unique for every node

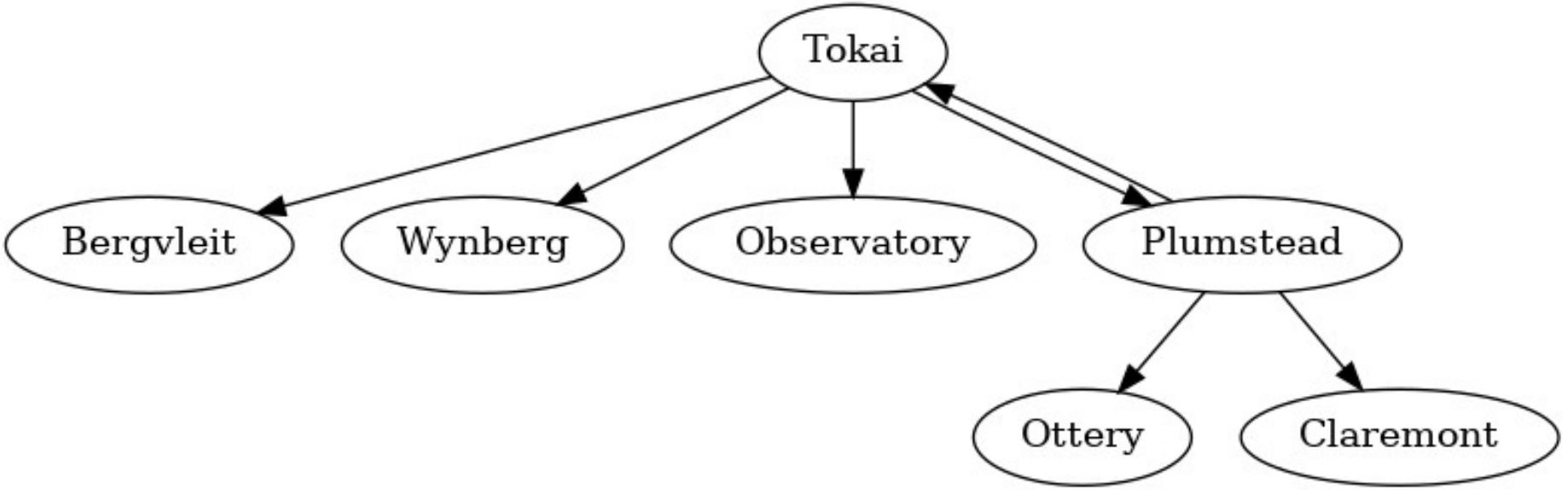
Search task


Rondebosch
27
Claremont
40
Plumstead
4
Ottery
4
Wynberg
15
Newlands
8
Constantia
24
Bishopscourt
5
Pinelands
18
Observatory
24
Tokai
2
5
4
Bergvliet
14
Kenilworth
35
Steenberg

Search task



Search task


Tokai
Bergvleit
Wynberg
Observatory
Plumstead
Ottery
Claremont

Search task

- Root is always the initial state
- Expansion: get all available actions for states
- Successor generation: use current state, actions, and transition model to obtain children
- Hierarchy: Each new child node is linked back to its origin

Search task

- Key difference between various uninformed algos:
  - Choosing one specific node to expand, while others set aside for later
- Openlist will store (un)expanded nodes
- (optional) List of nodes that have been been reached
- Every time expansion moves node from openlist to history/reached
- Children are added to openlist

Search task

- Graph representations can have clear cycles
- Tree-like representations duplicate nodes
- Cycle are special cases of redundant paths that can lead to infinite loops
- Graph Search: checks for and discards redundant paths
  - One needs to maintain explicit history data structure
- Tree-Like search does not check for redundancy
- Memory use for history vs. robustness/flexibility

Performance comparison

- What is the total time taken, measured by the number of states and actions explored? (Time Complexity)
- What is the maximum amount of memory required during the search process (Space Complexity)

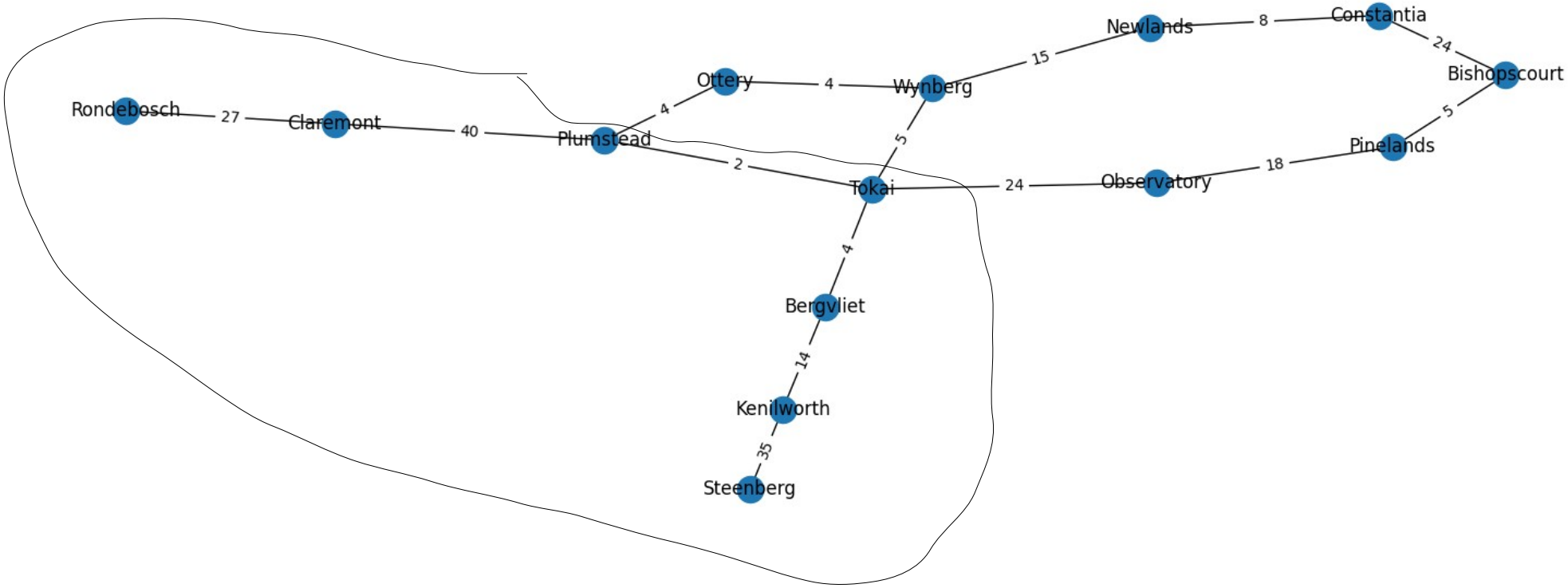
Performance comparison

- Are you guaranteed to find a solution, if one exists (Completeness)
- Will you always find the solution with the lowest possible path cost? (Cost Optimality)

Best-First Search

- Greedy best-first search is a hueristic-based algorithm
- Utility as pedagogical device

Best-first search



- Initial: Tokai, Goal: Rondebosch
- How does node expansion work?

Best-First Search

```
function EXPAND(problem, node) yields nodes
    s ← node.STATE
    for each action in problem.ACTIONS(s) do
        s' ← problem.RESULT(s, action)
        cost ← node.PATH-COST + problem.ACTION-COST(s, action, s')
        yield NODE(STATE=s', PARENT=node, ACTION=action, PATH-COST=cost)
```

Best-First Search

- The initial state is fixed, so you can process it first (mark it as visited)
- Expand the initial state, consider the children
- If one of the children is the goal, great!

Best-First Search

- The initial state is fixed, so you can process it first (mark it as visited)
- Expand the initial state, consider the children
- If one of the children is the goal, great!
- Each child either has not been visited, or you found a shorter path to it
- If that’s the case, add it to openlist and mark it as visited
- Important idea: how to order the openlist?
- Distance metric (e.g., Euclidean). Always start with closest nodes first!

Best-First Search

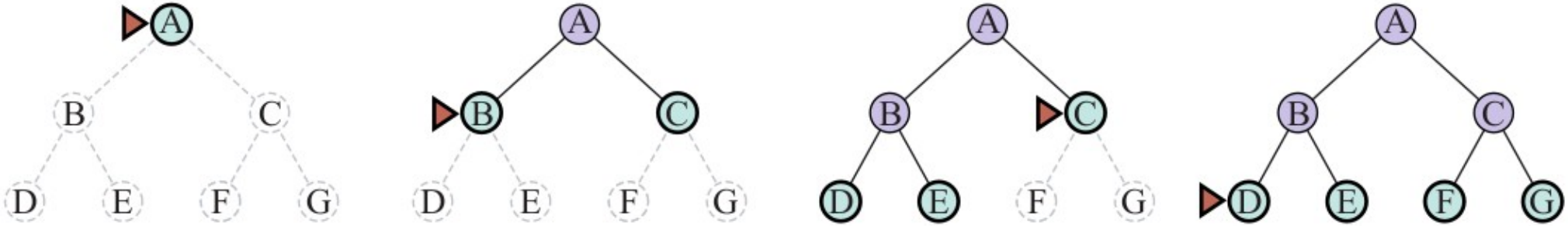
```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
    node ← NODE(STATE=problem.INITIAL)
    frontier ← a priority queue ordered by f, with node as an element
    reached ← a lookup table, with one entry with key problem.INITIAL and value node
    while not IS-EMPTY(frontier) do
        node ← POP(frontier)
        if problem.IS-GOAL(node.STATE) then return node
        for each child in EXPAND(problem, node) do
            s ← child.STATE
            if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
                reached[s] ← child
                add child to frontier
    return failure
```

Best-First Search

- Prioritizes nodes based on heuristic estimates
- Example: A* algorithm uses a priority queue
- Pros: Efficient for problems with heuristic guidance

Breadth-First Search (BFS)

- Explores all nodes at the current depth before moving to the next level
- Idea: since there is no heuristic function
  - Make the fewest number of actions to get to goal
  - The cost of actions needs be the same



Breadth-First Search (BFS)

```
function BREADTH-FIRST-SEARCH(problem) returns a solution node or failure
  node ← NODE(problem.INITIAL)
  if problem.IS-GOAL(node.STATE) then return node
  frontier ← a FIFO queue, with node as an element
  reached ← {problem.INITIAL}
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if problem.IS-GOAL(s) then return child
      if s is not in reached then
        add s to reached
        add child to frontier
  return failure
```

Breadth-First Search (BFS)

- Complete if every node has a finite number of children
- BFS is guaranteed to find the solution with the minimum number of actions
- Suppose that we have a solution with n actions.
- Why can’t there be a solution with fewer actions?
- Top down process considers:
  - Solution with no actions
  - Solution with 1 action
  - …
  - Solution with n – 1 actions

Uniform cost search

- Explores nodes in order of increasing cost
- The goal is to ‘remedy’ the constraint of uniform edge costs from BFS
- Use cumulative cost of path to identify cheap paths
- f(node) = node.path_cum_cost

```
function BEST-FIRST-SEARCH(problem, f) returns a solution node or failure
  node ← NODE(STATE=problem.INITIAL)
  frontier ← a priority queue ordered by f, with node as an element
  reached ← a lookup table, with one entry with key problem.INITIAL and value node
  while not IS-EMPTY(frontier) do
    node ← POP(frontier)
    if problem.IS-GOAL(node.STATE) then return node
    for each child in EXPAND(problem, node) do
      s ← child.STATE
      if s is not in reached or child.PATH-COST < reached[s].PATH-COST then
        reached[s] ← child
        add child to frontier
  return failure
```

Uniform cost search

- Time/Space complexity is $O(b^{1+\lfloor C / c \rfloor})$
- C = the cost of the optimal solution
- c = Small possible step cost

- Uniform-cost search is complete and is cost-optimal
  - Prioritization of low-cost path means: first solution is at least as low as the cost of any other node in the openlist
  - all paths are considered

Reading

- Sections 3.3 – 3.4.2

